

TP Sheet 1 - Introduction to robotics

Prepared by: Gonalo Leo

With contributions by: Professor Lus Paulo Reis and Professor Armando Sousa

Getting started

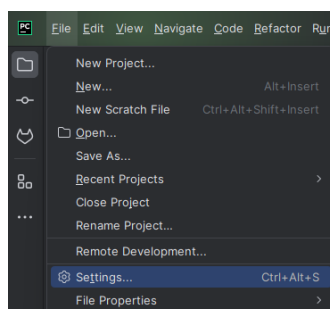
Begin by installing the Webots simulator according to the [official tutorial](#).

For all these exercises, use the Webots project provided in the course's [Github repository](#). All exercises use the same robot, in the same simulated environment. The robot is an [E-puck robot](#), a differential drive robot with two wheels and some sensors (including a set of distance sensors around the robot and a gyroscope). For these exercises, we added three extra sensors to the robot: a compass, a GPS and a LiDAR. The robot is located in a chessboard (known in Webots as a [RectangularArena](#)), where each tile has a length of 12.5 cm.

For each exercise, you will complete the “TODO” sections of a robot controller, written in Python. Each exercise is a .py file located in a sub-directory in the “controllers” directory of the Webots project.

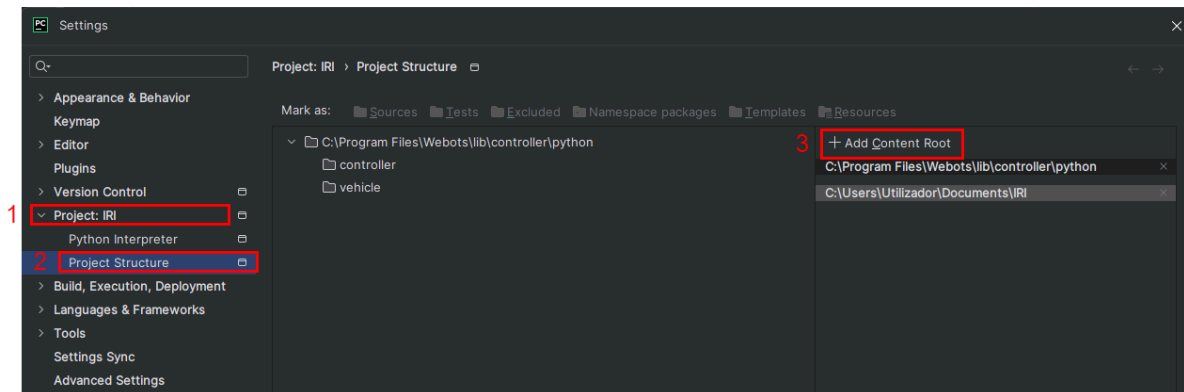
It is strongly suggested for you to use PyCharm as your IDE throughout all the practical classes. You will only need to set up the IDE once for all the exercises. In PyCharm, select “Open” and select the root directory of the course's [Github repository](#) (the one which has as direct children the “worlds” and “controllers” directories).

You then need to add the Webots Python libraries to your PyCharm project. To do this, go to “File” > “Settings”, as shown below.



A modal window will then open (as shown in the figure below). Select the option that starts with “Project: “ and then “Project Structure”. From there, in the right pane, click on “Add Content Root” and locate the directory in your computer with the Webots Python library. Within the Webots files, it is the one at “lib/controller/python”. You need to find out where Webots was installed in your computer:

- In Windows, the path is very likely: "C:\Program Files\Webots\lib\controller\python".
- In macOS, the path might be:
"/Applications/Webots.app/Contents/lib/controller/python".

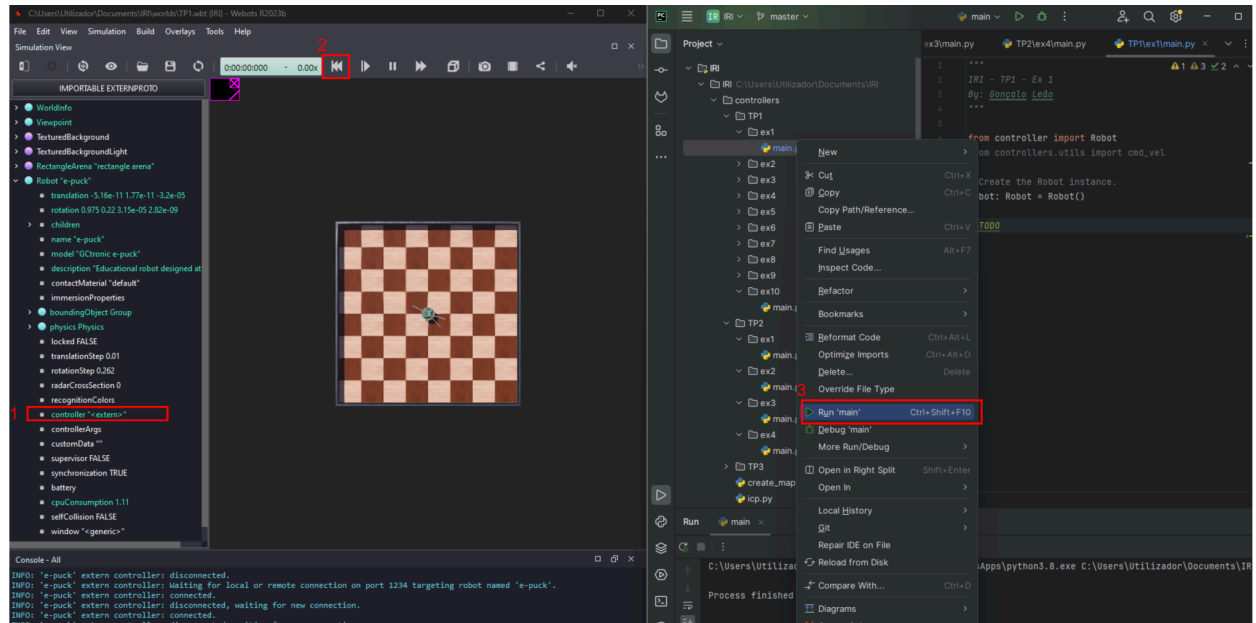


For the exercises of the first class, you need to import the world file "TP1.wbt". For this, in Webots, go to "File" > "Open World" and select "TP1.wbt", located in the "worlds" directory of the course's Github repository.

Once the IDE is set up. You should then be able to run your code as follows (as shown in the figure below):

- Ensure Webots is running with the provided .wbs file and that the "controller" field of the "e-puck" robot is set to "extern".
- Reset the simulation by pressing the button with the  symbol or pressing (ctrl + shift + T).
- Run the .py of the exercise you want to test from your IDE. The robot should now be running your code.

Suggestion: Split your screen between the Webots and Pycharm windows, for more comfortable programming. :) The image below shows a single-screen setup.



Troubleshooting

Q: I am getting an error related to WEBOTS_HOME.

A: You very likely need to set the WEBOTS_HOME environment variable. This is the path to the root folder of the Webots simulator you downloaded. You will need to find out where the folder is in your computer. In Windows, it is probably in "C:\Program Files\Webots". Note that this file is an "ancestor" folder of the one you need to provide to Pycharm so it can access the Webots libraries (i.e. the one with "lib/controller/python").

Q: I am getting an error saying that Python cannot be found.

A: You very likely need to add your Python executable to the PATH environment variable. You will need to find out where Python is on your computer.

Q: In the Pycharm console, I have the message "Cannot find any instance of Webots, retrying for another x seconds" and the robot is not moving.

A: Make sure Webots is running.

Q: In the Webots console, I am getting the error "refusing connection attempt from another extern controller".

A: Your simulator is paused. Press the  button.

Q: I changed a couple of parameters in Webots and I want to revert it back!

A: You simply need to press the  button to reload the world and select the “Discard” option in the window that pops up.

A - Actuators

Exercise 1 - Moving forward

Write a controller that makes the robot move continuously forward, with a constant linear velocity of 0.1 m/s.

To accomplish this, you can use the step function from the [Robot class](#) and the `cmd_vel` function that we already did for you in `controllers/utlis.py`. The `cmd_vel` function adjusts the speed of an e-puck robot's left and right wheel speeds according to a desired linear and angular velocity. This function adjusts these speeds according to the robot's wheel radius and the distance between the two wheels (known as the axle length). The linear velocity is in m/s and the angular velocity is in rad/s.

Exercise 2 - Making a circle

Write a controller that makes the robot move in a circle, in the counter-clockwise direction, with a linear velocity of 0.1 m/s and a circle radius of 12.5 cm (which corresponds to the length of a tile in the arena).

To compute the correct angular velocity, use the equation: $v = r \cdot w$, where v is the linear velocity (also known as the tangential velocity), r is the circle radius and w is the angular velocity.

Exercise 3 - Forward, stop and spin

Write a controller that makes the robot, in order:

- move forward for 1 second with a linear velocity of 0.1 m/s.
- stop for 1 second
- rotate around itself endlessly, in the clockwise direction, with an angular velocity of 180 degrees per second.

Exercise 4 - Back and forth

Write a controller that makes the robot, in an infinite loop, in order:

- move forward 2 tiles with a linear velocity of 0.1 m/s.
- stop for 1 second
- move backward 2 tiles with a linear velocity of 0.1 m/s.

Suggestion: implement the `move_forward` function in the `controller/utils.py` file, which you can then also use in some of the next exercises. The function makes a robot move forward, given the robot, desired distance and linear velocity as arguments.

Exercise 5 - Making a square

- a) Write a controller that makes the robot, in an infinite loop, move in a square-shaped trajectory, with the length corresponding to 2 tiles.

Suggestion: implement the `rotate` function in the `controller/utils.py` file, which you can then also use in some of the next exercises. The function makes a robot rotate around itself, given the robot, desired angle (theta) and angular velocity as arguments.

- b) The `cmd_vel` function you have been using was calibrated manually for this specific robot. The axle length was manually set to 57 mm, rather than 52 mm, as indicated in the [documentation](#). Try changing the axle length back to 52 mm, what happens to the trajectory?

Exercise 6 - Making a spiral

Write a controller that makes the robot rotate in a spiral trajectory with a radius that gradually decreases until the robot starts rotating around itself.

Exercise 7 - Making an inverted spiral

Write a controller that makes the robot rotate in a spiral trajectory, with increasing radius , until it reaches a radius of 2 squares.

B - Sensors

Exercise 8 - Playing with the LiDAR

- a) Write a controller that makes the robot print the XY coordinates of the points belonging to the point cloud captured by the LiDAR sensor. Take a look at the [LiDAR documentation](#).

Hint: Run the `print_devices.py` script to print in the console the name of all of the robot's sensors, so you know the string you need to pass to the `robot.getDevice` function to retrieve the sensor.

Suggestion: Call `lidar.enablePointCloud()` once in the beginning of your code and activate the option "View > Optional Rendering > Show Lidar Point Cloud" so that the LiDAR's rays are visible in Webots' GUI.

- b) Try changing the sensor's "horizontalResolution" (i.e. number of rays) and "fieldOfView" parameters. Confirm that the measured XY coordinates still make sense.

Exercise 9 - Playing with the compass

Write a controller that makes the robot endlessly rotate around itself and continuously print in the console the angle between the direction it is facing and the arena's x-axis vector (i.e. the vector in), using the compass sensor readings. Take a look at the [Compass documentation](#).

Exercise 10 - Playing with the GPS

Write a controller that makes the robot move in a circle (similarly to exercise 2) and continuously print in the console its XYZ position, using the GPS sensor readings. Take a look at the [GPS documentation](#).